

MATH 538 FINAL REPORT

Oscar Ramirez, Matthew Chen, Ann Kim, and Rahul Karki

December 19, 2025

The main idea behind Bayesian Backpropagation (BBB) is that all our weights of our neurons are represented as a “distribution” that we place on each of our parameters. Each of the weights potentially can have their own unique “distribution” in order to account for the uncertainty within our particular node. Placing prior distributions on network weights introduces regularization that can help reduce overfitting by controlling model complexity.

1 Introduction and Motivation

1.1 Limitations of Linear Models and the Need for Nonlinearity

Many real world patterns are far too complex to be captured by simple straight linear relationships. Medical risks, image features, financial behavior, and even learning outcomes often change in nonlinear and unpredictable ways. Because these relationships can bend, curve, or shift depending on the situation, traditional linear models can be too restrictive. They force the data into a rigid structure when the underlying reality is much more flexible.

Neural networks were developed to address this limitation. As powerful nonlinear models, they can learn intricate patterns directly from data and approximate functions of almost any shape. However, standard neural networks have one major weakness: they do not communicate uncertainty. A conventional neural network produces a single point prediction without providing uncertainty quantification. While this may be sufficient in low-risk settings, in applications such as medicine, autonomous systems, or financial prediction, the absence of credible intervals or posterior uncertainty can make it difficult to assess the reliability of predictions.

1.2 Motivation for Bayesian Neural Networks and Uncertainty Quantification

Bayesian Neural Networks (BNNs) provide a natural and principled solution. Instead of treating weights as fixed values, BNNs treat them as probability distributions. This allows the network to learn not only a prediction, but also the degree of uncertainty around that prediction. Rather than saying, “This is the answer,” a BNN says, “This is my answer, and here is how unsure I am.” This additional information helps us decide whether a prediction is reliable, whether more data is needed, or whether the model might be operating outside the range of what it has previously seen.

Although BNNs can be applied to any neural network architecture, including simple linear models, their benefits become especially clear in complex and nonlinear settings. In such settings, traditional neural networks provide only point estimates and do not quantify uncertainty in their predictions. By incorporating probability distributions over model parameters, Bayesian Neural Networks produce posterior predictive distributions that capture both expected outcomes and associated uncertainty, allowing model reliability to be assessed more

explicitly. This makes them extremely valuable in real world applications where understanding uncertainty is just as important as achieving accuracy.

Quantifying uncertainty is not only helpful but also often essential. A medical decision system should behave differently when it is confident compared to when it is unsure. A financial model should be able to separate ordinary variability from truly risky events. A system that interacts with the physical world, such as an autonomous vehicle, must know when the environment differs from anything it has encountered before. In these situations, modeling uncertainty directly helps prevent misleading confidence and reduces the chance of harmful decisions.

BNNs combine the expressive power of nonlinear neural networks with the ideas of Bayesian inference. This combination produces models that are flexible, transparent, and better aligned with the complexity of real data. They allow predictions to reflect both what the model has learned and what the model still does not know. For this reason, BNNs offer a compelling approach for modern applications where responsible and uncertainty aware prediction is essential.

2 Bayesian Neural Networks

2.1 Target Posterior and Posterior Predictive Distribution

The goal of using BNN is to calculate the expectation of the posterior distribution of our output y variable given our data and our weights for our neurons.

$$p(y \mid data, w) \tag{1}$$

$$p(w \mid data, x) = \frac{p(data \mid x, w) * p(w)}{p(data \mid x)} = \frac{p(data \mid x, w) * p(w)}{\int p(data \mid x, w) * p(w) dw} \tag{2}$$

2.2 Intractability of the Posterior in Neural Networks

Using the Bayes equation, we can see that which the denominator or the marginal likelihood of the data is intractable. By Bayes' rule, the posterior distribution is proportional to the product of the likelihood and the prior. Since the marginal likelihood is intractable, we work with this proportional form and approximate the posterior using variational inference.

$$p(w \mid x, data) \propto p(data \mid x, w) * p(w) \tag{3}$$

3 Variational Inference and Bayes by Backprop

3.1 Variational Approximation $q(w)$ to $p(w \mid D)$

However, in Bayesian neural networks, we want to approximate the intractable posterior distribution. In order to approximate the posterior distribution we use variational inference.

$$q(w \mid x) \approx p(w \mid x) \tag{4}$$

Source: (de Silva, 2021)

3.2 Evidence Lower Bound (ELBO) and KL Decomposition

Now we will use the Kullback-Leibler divergence which is the expected loss suffered as we adopt q as our model while the true distribution is p . First we will write out our equations and then prove it and even show Evidence of Lower bound (ELBO). In order to find the $q(w | x)$ that minimizes our KL divergence so that our approximate distribution is optimized to be as close to the true posterior distribution,

$$\theta^* = ARGmin_{\theta} [D_{kl}(q(w | x) || p(w | x))] \quad (5)$$

$$= ARGmin_{\theta} [\int q(w | x) * \log(\frac{q(w | x)}{p(w | x)}) dw] \quad (6)$$

$$= ARGmin_{\theta} [D_{kl}(q(w | x) || p(w)) - E_q[\log p(x | w)]] \quad (7)$$

Source: (Blundell et al., 2015)

Now we will find the evidence of the lower bound for the marginal Likelihood of $\log p(x)$

$$D_{kl}(q(w | \theta) || p(w | x)) = \int q(w | x) * \log(\frac{q(w | x)}{p(w | x)}) dw \quad (8)$$

$$= E_q[\log(\frac{q(w | x)}{p(w | x)})] \quad (9)$$

$$= E_q[\log(q(w | x)) - E_q[\log(p(w | x))]] \quad (10)$$

$$= E_q[\log(q(w | x)) - E_q[\log(\frac{q(w, x)}{p(x)})]] \quad (11)$$

$$= E_q[\log(q(w | x))] - (E_q[\log(p(w, x)) - E_q[\log(p(x))]]) \quad (12)$$

Using standard results from variational inference, the log marginal likelihood can be decomposed into the ELBO plus a Kullback–Leibler divergence term.

$$= \log(p(x)) - E_q[\log(p(w, x) - \log(q(w | x)))] \quad (13)$$

Now if we write the entire equation and change the order of terms we get

$$\log(p(x)) = D_{kl}(q(w | x) || p(w | x)) + E_q[\log(p(w, x) - \log(q(w | x)))] \quad (14)$$

One important property of KL divergence is that is can never be negative so, ≥ 0 ,

$$\log(p(x)) \geq E_q[\log(p(w, x) - \log(q(w | x)))] = ELBO \quad (15)$$

Which this gives us our Evidence Lower bound (ELBO) for our marginal Likelihood of $\log p(x)$. In order to optimize the KL Divergence we want to minimize it by maximizing the ELBO term. But, in order to do so we have to optimize our parameters for q and for p which we will call ϕ and θ . Which we will do this by finding the gradient for both ϕ and θ to maximize the ELBO.

Which leads us to taking the gradient of the ELBO in respect to θ and ϕ :

$$L(x) = -E_{q\phi}[\log(\frac{p_\theta(w, x)}{q_\phi(w|x)})] \quad (16)$$

$$\nabla_{\theta, \phi} L(x) = -\nabla_{\theta, \phi} E_{q\phi}[\log(\frac{p_\theta(w, x)}{q_\phi(w|x)})] \quad (17)$$

Which let us first take the gradient in respect to θ .

$$= \nabla_\theta (E_{q\phi}[\log(p(w, x) - \log(q(w|x)))] \quad (18)$$

$$= \nabla_\theta (\int_w q_\phi(w|x) [\log(p(w, x) - \log(q(w|x)))] dw \quad (19)$$

We pass the gradient with respect to theta to be passed in our f(x) part of the definition of the expectation

$$= (\int_w q_\phi(w|x) \nabla_\theta [\log(p(w, x) - \log(q(w|x)))] dw \quad (20)$$

$$(E_{q\phi} \nabla_\theta [\log(p(w, x) - \log(q(w|x)))] \quad (21)$$

Which we simply to be,

$$\approx 1/L * \sum_{i=1}^L \nabla_\theta \log(p(w, x)) \quad (22)$$

Which we can solve using Monte Carlos sampling techniques. Now we have to find the gradient of the ELBO in respect to ϕ .

$$\nabla_\phi (E_{q\phi}[\log(p(w, x) - \log(q(w|x)))] \quad (23)$$

$$= \nabla_\phi (\int_w q_\phi(w|x) [\log(p(w, x) - \log(q(w|x)))] dw \quad (24)$$

We to remember the basic idea of gradients is we are taking derivatives in respect to our parameter dimensions θ and ϕ . Since both the ELBO and non-ELBO functions within the integral both are affected by the derivative in respect to ϕ , we must do the Product Rule of Differentiation which gives us,

$$= (\int_w q_\phi(w|x) \nabla_\phi [\log(p(w, x) - \log(q(w|x)))] dw + (\int_w [\log(p(w, x) - \log(q(w|x)))] \nabla_\phi q_\phi(w|x) dw \quad (25)$$

Which the first equation can simplify to

$$= E_{q\theta}(w|x) [\nabla_\phi ELBO] + \int_w [ELBO] \nabla_\phi q_\phi(w|x) dw \quad (26)$$

Now we can approximate the expectation part of the equation; however, we can not approximate the ϕ gradient of q.

3.3 Reparameterization Trick for Gradient-Based Optimization

But another way we can approximate it is if we Re-parametrize q with an equivalent distribution that is not parametrized by ϕ .

Let w equal some function g with parameters ϕ, x, ϵ :

$$w = g(\phi, x, \epsilon) \quad (27)$$

Where ϵ follows some sort of probability distribution, $\epsilon \sim p(\epsilon)$

We want our ϕ, x parameters to be deterministic while ϵ to have all the stochasticity/randomness, similar to the concept of linear regression models. Doing this we can now get the second part of that equation which is the expectation with respect to $p(\epsilon)$ instead of ϕ

$$L_4(x) = -E_{p(\epsilon)}[\log(p(w, x) - \log(q(w|x)))] \quad (28)$$

Which at this point we can generalize the gradient solution to be

$$\nabla_{\phi\theta} L(x) \approx -1/L \sum_{i=1}^L \nabla_{\phi\theta} [\log(p(w, x) - \log(q(w|x)))] \quad (29)$$

Assuming our weights have a Gaussian-normal distribution we can actually find the analytic closed form for the KL divergence. First we have to use the same Reparameterization assumptions we made before, Let w equal some function g with parameters ϕ, x, ϵ

$$w = g(\phi, x, \epsilon) = N(\mu, \sigma) \quad (30)$$

$$\epsilon \sim p(\epsilon) \quad (31)$$

Which ϵ commonly follows a gaussian normal distribution. The prior distribution is defined as a multivariate unit gaussian distribution

$$p_\theta(w) = N(0, I) \quad (32)$$

where μ and σ are calculated using a deterministic model that depends on x

$$(\mu, \sigma) = f(x) \quad (33)$$

Now let us write the Probability Distribution Function of a Gaussian-Normal distribution which is

$$N(w|\mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} * e^{-\frac{1}{2}*((w-\mu)/\sigma)^2} \quad (34)$$

Now we can plug the Gaussian-Normal distribution within the KL-divergence formula,

$$D_{kl}(q(w|\theta)||p(w)) = \int q(w|x) * \log\left(\frac{q(w|x)}{p(w)}\right) dw \quad (35)$$

$$= \int q(w|x) * \log\left(\frac{1}{\sigma\sqrt{2\pi}} * e^{-\frac{1}{2}[(w-\mu)/\sigma]^2 - w^2}\right) dw \quad (36)$$

Simplifying we get,

$$= -1/2 \int q(w|x) * \log(\sigma^2 - w^2 + 1/\sigma^2(w - \mu)^2) dw \quad (37)$$

$$= -1/2 * \log(\sigma^2) \int q(w|x) dw + 1/2 \int q(w|x) w^2 dw - 1/2\sigma^2 \int q(w|x)(w - \mu)^2 dw \quad (38)$$

First, if $q(w|x)$ is a valid probability density function that it must integrate to one.

$$\int q(w|x) dw = 1 \quad (39)$$

Second, the definition of variance is equal to,

$$Var(w) = \int q(w|x)(w - \mu)^2 dw = \sigma^2 \quad (40)$$

Third, using the expectation formula for variance:

$$Var(w) = E_q[w^2] - E_q[w]^2 \quad (41)$$

$$\int q(w|x) w^2 dw = E_q[w^2] \quad (42)$$

$$= Var(w) + E_q[w]^2 = \sigma^2 + \mu^2 \quad (43)$$

Now we can simplify our KL divergence formula to be

$$D_{kl}(q(w|\theta)||p(w)) = -1/2[\log(\sigma^2) - (\sigma^2 + \mu^2) + 1] \quad (44)$$

Which now we have one component of the EVIDENCE OF LOWER-BOUND (ELBO)

$$ELBO = E_q[\log(p(w, x) - \log(q(w|x)))] \quad (45)$$

$$= E_q[\log(p(w, x))] - E_q[\log(q(w|x))] \quad (46)$$

$$= E_q[\log(p(x|w) * p(w))] - E_q[\log(q(w|x))] \quad (47)$$

$$= E_q[\log(p(x|w))] + E_q[\log(p(w))] - E_q[\log(q(w|x))] \quad (48)$$

If we group the last two terms of the equation we can simply to be the formula for KL divergence

$$= E_q[\log(p(x|w))] - D_{kl}(q(w|x)||p(w|x)) \quad (49)$$

3.4 Training Objective in Bayes by Backprop

At this point we can use Monte Carlo Sampling techniques to get an approximation of the expectation of $\log p(x|w)$ with respect to q .

$$p(x|w) \sim N(\mu_3, \sigma_3) \quad (50)$$

Our goal is to maximize the expectation term by minimizing, in order to maximize ELBO in order to get the best approximation as possible. As you can see above having a small σ_3 and minimizing the squared terms

with μ_3 will do exactly that.

4 Experiment 1: Concrete Compressive Strength Regression

4.1 Data Description and Preprocessing

We will be using Bayesian Neural Network as a regression model giving our featured variables normal distributions to get the best predictions. The dataset we will be working with originates from Taiwan in 2007 which can be found on the UCI dataset Repository. We have 1030 observations with 9 different variables of information amount of cement (kg/m^3), blast furnace (kg/m^3), fly ash (kg/m^3), water (kg/m^3), super plasticizer (kg/m^3), coarse aggregate (kg/m^3), fine aggregate (kg/m^3), amount of days for cement to age, and concrete compressive strength (MPa). Every variable except for age variable is some sort of material variable they included to make concrete. Our goal will be to get a better regression model using Bayesian Neural Networks to model concrete compressive strength than using multiple linear regression. Recall, that Neural Networks are extremely good models for nonlinear data, in cases of linear data regression models tend to perform just as well as Bayesian Neural Networks. Two major ways we can detect the nonlinearity of our featured variables against our dependent variables. First, we can examine each of the scatterplot of each featured variable versus the dependent variables

4.2 Baseline: Multiple Linear Regression Diagnostics

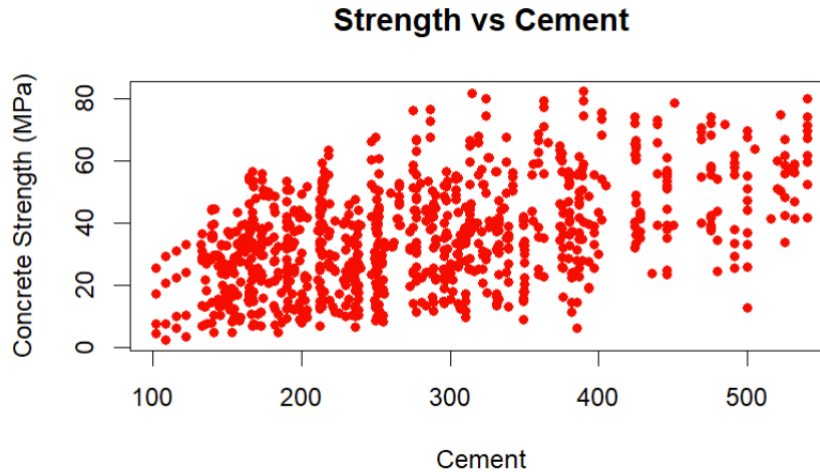


Figure 1: Scatterplot of Concrete Compressive Strength versus Cement

When cement is towards the lower amount we can see as we increase it seems to be taking on a megaphone increase structure. Until, 300 (kg/m^3) of cement where it seem to level off a bit. Clearly there seems to be evidence of a non-linear relationship between Cement amount and Concrete Strength. However, the easiest way we can detect non-linear relationships in our model is running the residuals versus fitted plot. These method just requires a single plot rather plotting every single scatterplot for each featured variable. Indicators to look for would be if there is not constant variance, shape or pattern of points, or clustering of points.

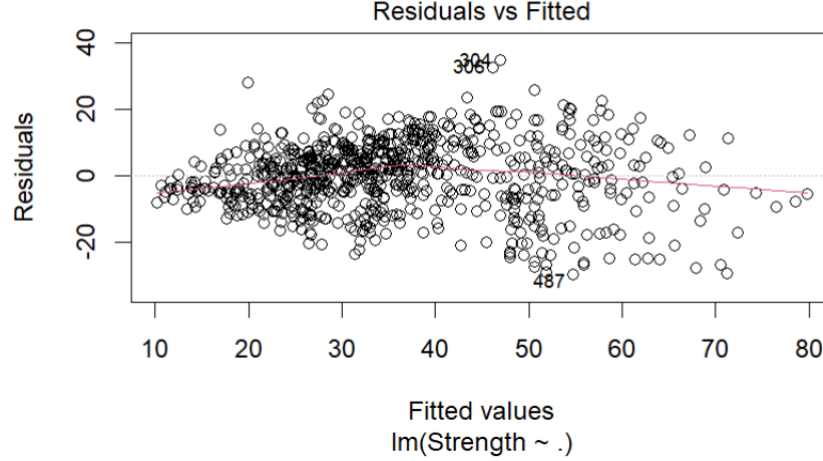


Figure 2: Residuals versus Fitted Plot for MLR

The LOESS smooth in the residuals versus fitted plot deviates noticeably from a horizontal line, indicating systematic structure in the residuals. In addition, the spread of residuals increases at higher fitted values, suggesting heteroscedasticity, and a clustering of points appears in the mid-range of fitted values. Together, these patterns provide evidence that the linearity and constant variance assumptions of the multiple linear regression model may be violated, supporting the presence of nonlinear relationships between predictors and the response.

4.3 Bayesian Neural Network Model and Training

Now that we have evidence of nonlinear relationship between featured and dependent variables. We are going to begin the coding process of Bayesian Backpropagation in python where we are going to use python library pandas, numpy, Pytorch, sklearn, and Matplotlib. We then proceed to our load up our dataset that we are working on which involves exploring concentrate compression strength depending on different material that they included within the concrete as well as how long they allowed it to dry. Using sklearn train/split function to perform cross validation in splitting up our dataset in 80 percent training and 20 percent training to not have any bias in our model. Using sklearn pre-processing function we want to standardized our training and testing datasets for our independent variables in respect to our training dataset. Doing this will make numerical computation easier and faster because typically Neural Networks are often make computationally heavy.

Now to use our vectors within the PyTorch library functions we need to make our testing and training datasets are made into tensors. Using Pytorch utilities we get TensorDataset to pair our tensors together and Dataloader to randomly shuffle then batch our data. Now, we begin creating the neural network parameter features where in it we are going to define our parameters mu's and epsilon having a normal distribution. Our rho is going to have a uniform distribution form -5 to -4 because once we apply the soft plus function onto the distribution going forward we will get a positive standard deviation value between 0.0067 and 0.018. Within the forward part of Pytorch neural network function our mu's, sigma's, and epsilons will help us get values for our weight and bias values where we get a linear combination using the weight multiplied by the data and adding the bias value we get the value z for our next layer. We also define in each layer the kl loss within the bias and weight equations which we need. Now we will define the amount of layers within the neural network which we will define as being one input layer with each featured variable having a neuron, 2 hidden layers with each having 64 neurons, and one output layer with 1 neuron. Note that in the hidden layers we will apply RELU at

the end of the z value calculations. RELU is a function that makes negative values a zero but does not change positive values. We also will keep track of the kl loss from each neural network layer to get the total. At this point we have defined our parameters and layers for neural network we know are going to train the model. Now we are going to train the entire neural network 50 times where in each step we are going to calculate the KL loss from each time and divide it by the 32 which is the number of batches of data that python is doing the calculation on every time. We are going to get the prediction using the training model and calculate the MSE. Then, using the pytorch optimizer package called Adam we first get the loss for the neural network then, calculate the gradients of our 4 parameters. Then using optimizer.step to update our parameters to get a lower loss or (-ELBO) value. From there we have trained our model now perform Monte Carlos sampling to get a distribution of our predicted values of the algorithm, we have to make sure to get our variables into numpy objects from tensors to easier explore the arrays after using the pytorch library/format. We have to make sure to unstandarize the predictions values and using sklearn library functions we can easy calculate the MSE, RMSE, and rsquared metrics for our Bayesian BackPropagation model. Now using the Concrete Compressive dataset where our independent variables are the materials included and how long in days the concrete was allowed to dry we applied our BBB regression model.

4.4 Predictive Performance and Uncertainty Quantification

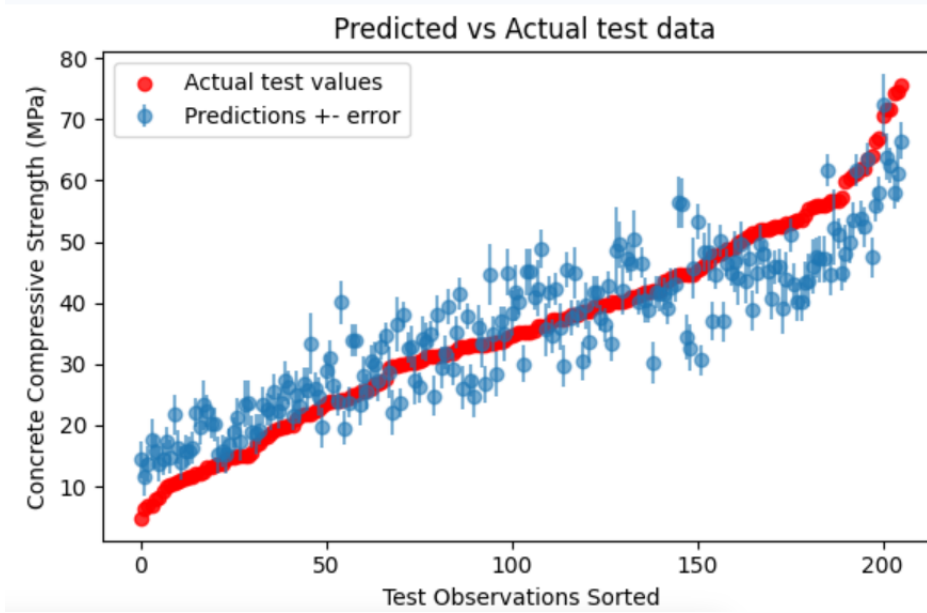


Figure 3: Predicted vs Actual test dataset

Now for our Bayesian Neural Network we got a MSE of 48.44 , RMSE of 6.96, and R^2 value of 0.812 which explains 81.2 percent of the variation in the concrete compressive strength. Which we actually include the error bars for every single mc sample so that we can see the potential spread of all 2000 possible values we sampled. While nonlinear patterns were present in the data, they were moderate rather than extreme, motivating comparison between linear and nonlinear modeling approaches. So our next goal is to compare BBB model performance with non-linear oscillatory data compared to Gaussian processes.

5 Experiment 2: Oscillatory Drone Dataset and GP Comparison

Now we decided to use a non-linear oscillatory dataset that was provided from The Defense Advanced Research Projects Agency (DARPA) to compare how well Bayesian Backpropagation Neural Networks and Gaussian Processes model the data. We decided on only using a subset of two variables where the first is the distance (km) the missile had to the test drone following it from until intercepting the enemy drones. While the second is the error in meters that the drone tracking the missile had from the enemy drone.

5.1 Bayesian Neural Network Architecture and Training

What we decided to do for our BNN was to use the similar structure for our concrete dataset. Except this time we allowed our distributions for the normal and uniform distributions to be a bit bigger since we have really non-linear oscillation data for our weight, and bias terms. Also we decided instead of doing two hidden layers we decided to do 3 where our first hidden layer is 64 neurons, our second hidden layer is 64 neurons, and our third hidden layer was 32 neurons. In order to allow our reparameterization equations to take on oscillatory behavior we decided to apply the hyperbolic tangent function. We did this for each of the resulting z values within our hidden layers because it gives us an oscillation like curves. A hyperbolic tangent function according is a scaled and shifted logistic sigmoid (Beck and Eldan, 2020). One major advantage in using hyperbolic tangent is that since it is bounded from -1 to 1. However, the 95 percent credible intervals will plateau within peaks because the gradient of our hyperbolic tangent is zero. Since our range of values are constrained from -1 to 1, our data does not have increasing variance therefore giving us the Plateau effect. We used the same learning rate of 0.001 as before so that when calculating our gradients in respect to our parameters. Which the change is small for every epoch that we run later. However, we decided to make kl weight smaller because the smaller we make this the more our regression line will look similar to our new data. The bigger we make kl weight the more our regression line will look similar to our prior distribution for our parameters. The drawback of having smaller kl weight is that we get bad 95 percent credible bands and having big kl weights gives us bigger credible intervals of uncertainty.

5.2 Predictive Results and Uncertainty Behavior

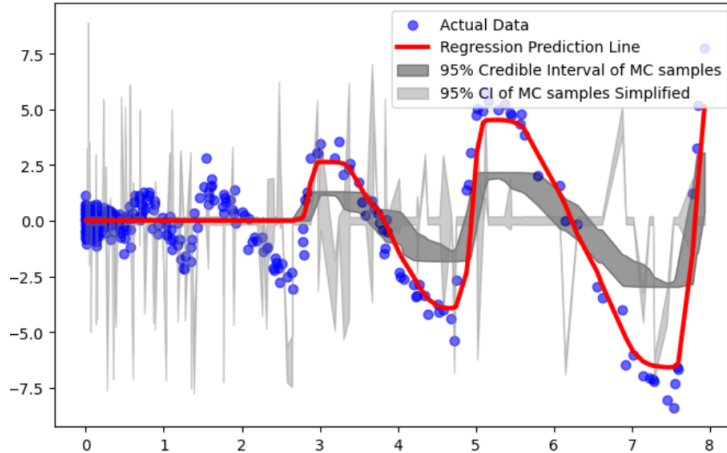


Figure 4: Scatterplot of actual data versus BBB Neural Network model with 95 percent CI

For example, using our Bayes Backpropagation algorithm in python. We get that our best model for predictive accuracy had an MSE of 0.5850, RMSE 0.7648, and R^2 value of 0.8857. The model explains 88.57 percent of the variation of the error between the missile and the enemy drone that we want to intercept for (DARPA). The red regression prediction line is our mean of the MC samples so in areas where our 95 CI interval does not include the observed testing data set. Which we would call our prediction as being overconfident which is not ideal. In non-peak areas we can see that the 95 percent credible interval of our posterior MC samples and our predicted regression line properly lies in between. Giving us what seems to be some places where our predictions are not overconfident.

Considering the data set was relatively small with only 1,400 observations, the GP outperformed the Bayesian model because it utilizes the suboptimal hyperbolic tangent activation function within our hidden layers. In this case, we would want to explore other hidden layer functions that better model increasing oscillations, potentially using the wave activation function which is a polynomial exponential function.

5.3 Gaussian Process Regression and Scalability Discussion

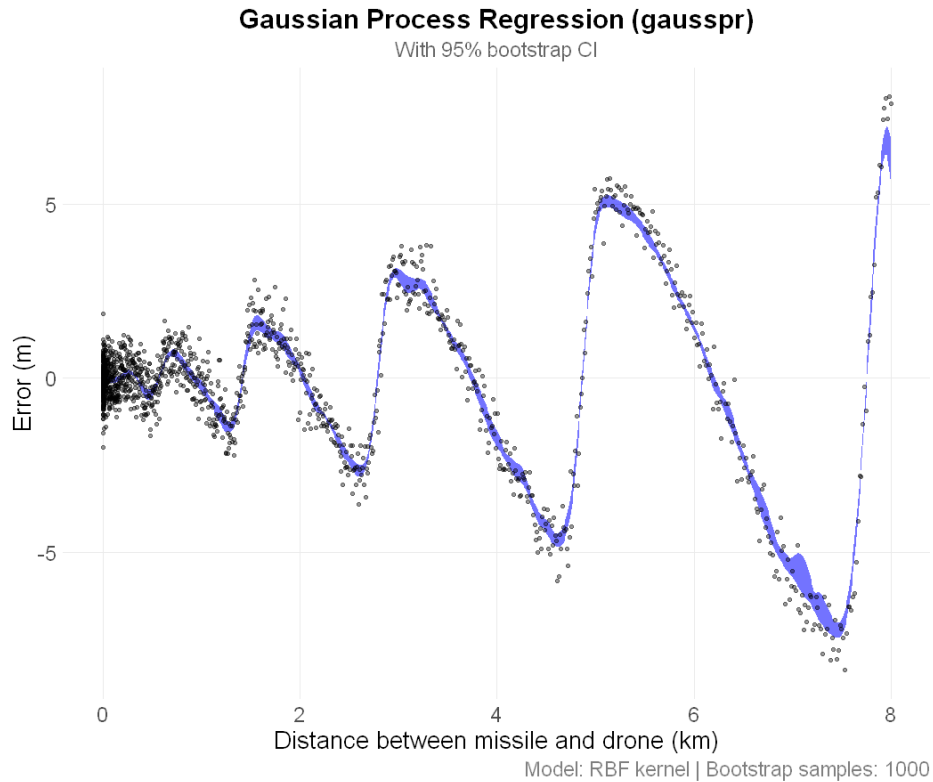


Figure 5: Scatterplot of actual data versus Gaussian Process Prediction model with 95 percent CI

For the Gaussian Process (GP) model, we chose a radial basis function (Gaussian) kernel because it can handle the complex, nonlinear data that we are feeding the model through. The noise variance parameter we selected was 0.01 which means a tighter fit since the values follow a recognizable pattern with little noise. In order to formulate the CI, a bootstrap was performed over 1000 iterations and the results were then filtered. As we can see, the 95% CI follow the pattern of the complex, oscillatory behavior of the drone error measurements. When validating with the testing data, the metrics were slightly better than the Bayesian neural network with a R^2 of 0.94, MSE of 0.29, and RMSE of 0.54. Since this is a relatively small dataset, the GP was expected

to handle the data better but the BNN is more suitable for much larger datasets which is more practical in real-world settings. As the dataset grows, the GP will require much more computational time since it scales $O(n^3)$; thus, the BNN should be utilized. Furthermore, BNNs are more flexible, capture more complex patterns, and handle higher dimensional datasets better than the GP model.

6 Fully Bayesian Neural Network in Stan via HMC

In addition to the variational Bayesian neural network implemented in Python, we also applied a fully Bayesian neural network using RStan to the same nonlinear oscillatory dataset. While the Python implementation relied on variational inference to approximate the posterior distribution of the network parameters, the RStan approach performs full Bayesian inference by directly sampling from the posterior distribution using Hamiltonian Monte Carlo (HMC) with the No-U-Turn Sampler (NUTS).

6.1 Model Specification and Priors

For this model, we again used distance (km) as the input variable and tracking error (meters) as the response. The neural network architecture consisted of a single hidden layer with $H = 3$ hidden units and a hyperbolic tangent activation function. Although this architecture is much smaller than the deep network used in the Python implementation, it was chosen deliberately to balance model flexibility and computational cost. Increasing the number of hidden units substantially increases the dimensionality of the parameter space and leads to significantly longer runtimes under HMC sampling. The goal of this section was therefore not to optimize network architecture, but to demonstrate exact Bayesian inference for a nonlinear neural network and to compare its behavior with variational methods. All input variables were standardized using statistics computed from the training set only. Weakly informative priors were placed on the network weights and biases, with hierarchical scale parameters controlling overall weight magnitude.

6.2 Sampling Strategy and Computational Considerations

Posterior sampling was performed using four Markov chains, each with 4,000 iterations and a warmup period of 2,000 iterations. The total runtime for the initial model fit was approximately 40–50 minutes, reflecting the substantially higher computational cost of full Bayesian inference relative to variational approaches.

The computational cost of this approach arises primarily from the need to evaluate gradients of the log posterior at each HMC step, with performance becoming increasingly sensitive to the dimensionality of the parameter space. While the No-U-Turn Sampler automatically adapts trajectory lengths, careful tuning is still required to ensure stable sampling behavior. In our implementation, conservative settings were used to prioritize reliable posterior exploration over runtime efficiency.

6.3 Posterior Predictive Results and Comparison to Linear Regression

Figure 6 illustrates the posterior predictive mean function of the Bayesian neural network along with 95 percent credible intervals. The model successfully captures the oscillatory structure present in the data, while the widening uncertainty bands in regions with sparse observations reflect increased posterior uncertainty.

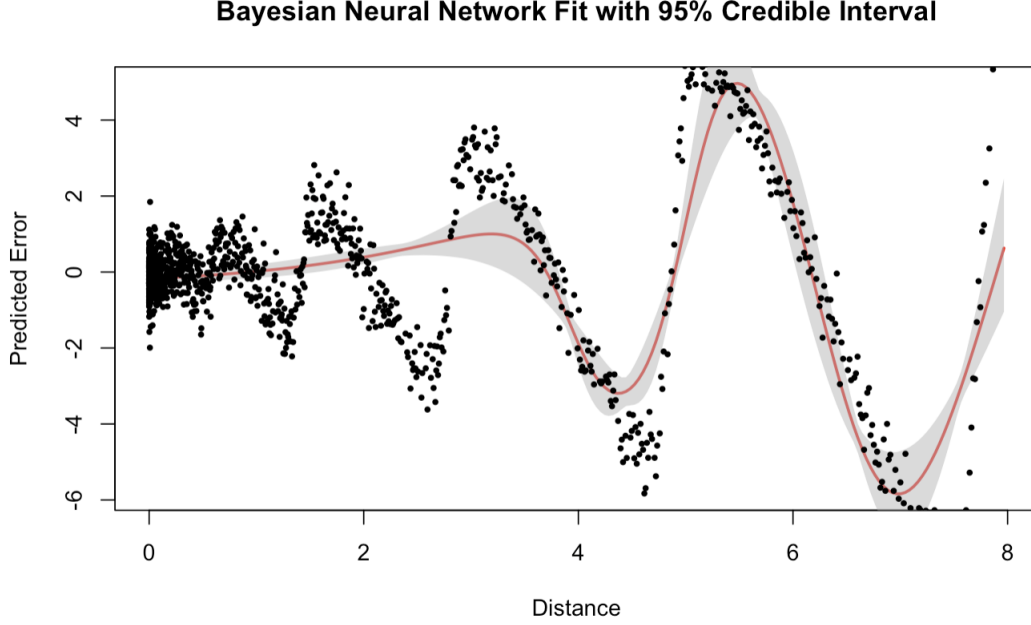


Figure 6: Scatterplot of Error (m) versus True Distance between Missile and Drone (km) with Bayesian Neural Network fit with 95 percent CI

Posterior predictive inference was conducted by propagating posterior samples of the network parameters through the neural network to obtain predictive distributions for the test data. From these samples, posterior means and 95 percent credible intervals were computed. Figure 6 shows the nonlinear posterior mean function along with its associated credible interval band, illustrating that the model successfully captures the oscillatory structure present in the data while also providing principled uncertainty quantification. To assess predictive performance, we compared the Bayesian neural network to a linear regression baseline. The linear model achieved a test RMSE of 2.307, while the RStan Bayesian neural network achieved a lower test RMSE of 1.549, demonstrating a clear improvement in predictive accuracy due to nonlinear modeling.

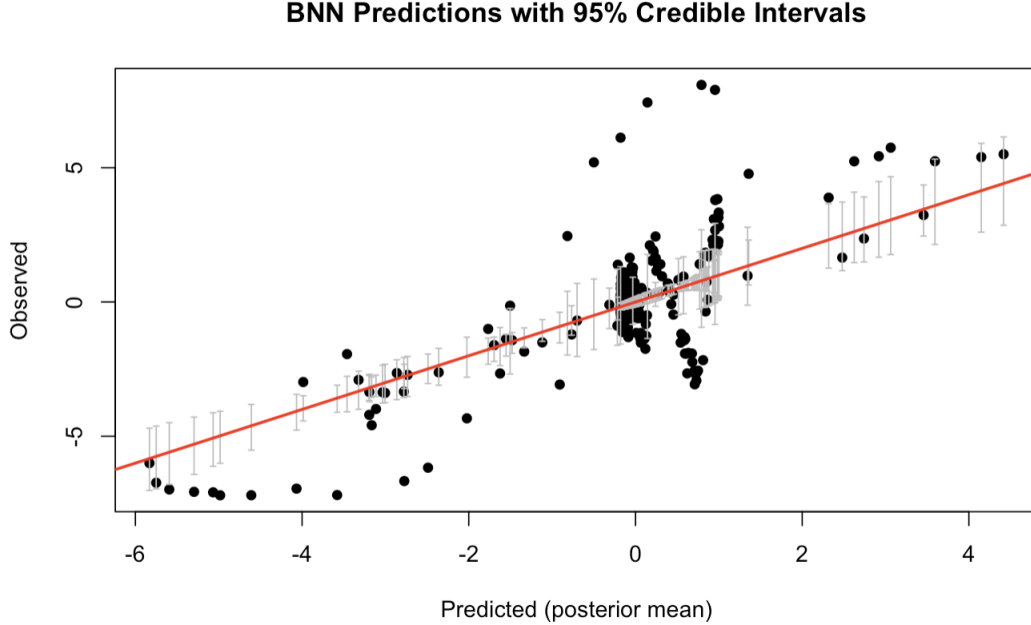


Figure 7: Actual values versus BNN predicted values

Figure 7 shows that posterior mean predictions closely align with the observed test values, while the credible intervals appropriately widen in regions of higher uncertainty, supporting the improved predictive performance of the Bayesian neural network relative to linear regression. The posterior predictive mean plotted against the observed values illustrates the predictive performance of the Bayesian neural network. The fitted regression line shows strong alignment between posterior mean predictions and observed outcomes, while the vertical 95 percent credible intervals quantify predictive uncertainty. Wider intervals are visible for more extreme predictions, indicating increased uncertainty in regions with fewer supporting observations. Although the predictive accuracy is lower than that achieved by the variational Bayesian neural network and Gaussian process models, the RStan approach provides a more faithful representation of posterior uncertainty, since it samples directly from the true posterior distribution rather than relying on an approximation. Overall, this example highlights the tradeoff between computational efficiency and inferential accuracy. Variational Bayesian neural networks offer fast and scalable approximate inference, while Hamiltonian Monte Carlo provides more accurate posterior uncertainty at the cost of substantially increased computation time. For small to moderate datasets where uncertainty quantification is critical, full Bayesian neural networks implemented via HMC can be a powerful modeling tool. Also including a relatively useful hidden layer activation function that better represents the nonlinear behavior our data can massively increase the model performance and uncertainty bands of the Bayesian Neural Network.

7 Conclusion

In this project, we investigated Bayesian neural networks as an extension of standard neural networks that explicitly accounts for uncertainty in model parameters and predictions. Rather than treating network weights as fixed quantities, Bayesian neural networks place probability distributions on weights and use Bayesian inference to obtain posterior predictive distributions. Because the exact posterior distribution $p(w \mid D)$ is generally

intractable for neural networks, we implemented Bayes by Backprop (BBB) using variational inference, where a tractable distribution $q_\phi(w)$ is optimized by maximizing the evidence lower bound (ELBO), balancing data fit with KL-based regularization. We applied this approach to two regression problems with different structural characteristics.

For the UCI Concrete Compressive Strength dataset, diagnostics from a multiple linear regression baseline suggested moderate nonlinearity and heteroscedasticity. The BBB model provided improved predictive performance while also producing meaningful uncertainty estimates through Monte Carlo forward passes. For the oscillatory drone dataset, we modified the network architecture to better capture periodic behavior and compared BBB to Gaussian process regression (Kunc and Kléma, 2024). In this setting, the Gaussian process achieved stronger predictive accuracy and uncertainty bands that closely followed the oscillatory structure, which is consistent with the flexibility of GPs in smaller datasets. At the same time, we noted that exact GP inference scales cubically with the number of observations, highlighting practical limitations as data size increases. To further examine posterior inference, we implemented a fully Bayesian neural network in Stan using Hamiltonian Monte Carlo (NUTS). Although this approach required substantially greater computational effort and a simpler network architecture, it provided a more accurate characterization of posterior uncertainty compared to variational approximations. Taken together, these results illustrate a fundamental tradeoff observed throughout the analysis: variational methods such as Bayes by Backprop offer computational efficiency and scalability, while HMC-based approaches provide higher-quality posterior inference at increased computational cost. Overall, this study demonstrates how different Bayesian modeling strategies balance flexibility, uncertainty quantification, and computational feasibility. Future work could explore alternative activation functions within the hidden layer, prior/parameter specifications, likelihood models, or network architectures, as well as extensions better suited for strongly periodic or higher-dimensional data.

8 Code

8.1 Bayesian Linear Parameters

```
class BayesianLinear(nn.Module):
    def __init__(self, in_features, out_features):
        super().__init__()
        # Mean of weight and bias initialized with small Gaussian noise
        self.weight_mu = nn.Parameter
            (torch.Tensor(out_features, in_features).normal_(0, 0.1))
        self.bias_mu = nn.Parameter
            (torch.Tensor(out_features).normal_(0, 0.1))

        # Rho parameter for weight and bias,
        # transformed via softplus to get positive std
        self.weight_rho = nn.Parameter
            (torch.Tensor(out_features, in_features).uniform_(-5, -4))
        self.bias_rho = nn.Parameter
            (torch.Tensor(out_features).uniform_(-5, -4))

        # Softplus ensures positive standard deviation
        self.softplus = nn.Softplus()

    def forward(self, x):
        # Convert rho to sigma using softplus
        weight_sigma = self.softplus(self.weight_rho)
        bias_sigma = self.softplus(self.bias_rho)

        # Sample epsilon from standard normal
        weight_eps = torch.randn_like(self.weight_mu)
        bias_eps = torch.randn_like(self.bias_mu)

        # Reparameterization trick: sample weights and biases
        weight = self.weight_mu + weight_sigma * weight_eps
        bias = self.bias_mu + bias_sigma * bias_eps

        # Apply linear transformation
        return F.linear(x, weight, bias)
```

8.2 KL Divergence

```
def kl_loss(self):
    # Convert rho to sigma
    weight_sigma = self.softplus(self.weight_rho)
    bias_sigma = self.softplus(self.bias_rho)
```



```

# KL divergence between posterior and standard normal prior
weight_kl = (torch.log(1.0 / weight_sigma) +
              (weight_sigma**2 + self.weight_mu**2)/2 - 0.5).sum()
bias_kl = (torch.log(1.0 / bias_sigma) +
           (bias_sigma**2 + self.bias_mu**2)/2 - 0.5).sum()

# Total KL loss
return weight_kl + bias_kl

```

(Kumar, nd)

8.3 Bayes By Backpropagation architecture example 1

```

class BayesByBackpropNN(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        # Three Bayesian linear layers
        self.b1 = BayesianLinear(input_dim, 64)
        self.b2 = BayesianLinear(64, 64)
        self.b3 = BayesianLinear(64, 1)

        # ReLU activation
        self.relu = nn.ReLU()

    def forward(self, x):
        k1 = 0 # Accumulate KL loss

        # First layer + KL
        x = self.relu(self.b1(x))
        k1 += self.b1.kl_loss()

        # Second layer + KL
        x = self.relu(self.b2(x))
        k1 += self.b2.kl_loss()

        # Output layer + KL
        x = self.b3(x)
        k1 += self.b3.kl_loss()

        return x, k1

```

8.4 Bayes By Backpropagation architecture example 2

```

class BayesByBackpropNN(nn.Module):
    def __init__(self, input_dim):
        super().__init__()

```

```

self.b1 = BayesianLinear(input_dim,64)
self.b2 = BayesianLinear(64,32)
self.b3 = BayesianLinear(32,1)
self.Tan= nn.Tanh()

def forward(self,x):
    k1=0
    x= self.Tan(self.b1(x))
    k1 += self.b1.kl_loss()
    x= self.Tan(self.b2(x))
    k1 += self.b2.kl_loss()
    x= self.b3(x)
    k1 += self.b3.kl_loss()
    return x,k1

```

8.5 Training Loop

(PyTorch Contributors, nd)

```

criterion = nn.MSELoss()  # Reconstruction loss
optimizer = optim.Adam(model.parameters(), lr=0.001)
epochs = 50
kl_weight = 1.0 / len(train_loader)  # Scale KL by batch count

for epoch in range(epochs):
    model.train()
    running_loss = 0

    for x_batch, y_batch in train_loader:
        x_batch = x_batch.to(device)
        y_batch = y_batch.to(device)

        optimizer.zero_grad()

        # Forward pass returns prediction and KL loss
        pred, kl = model(x_batch)

        # ELBO = MSE + scaled KL
        loss = criterion(pred, y_batch) + kl_weight * kl
        loss.backward()
        optimizer.step()

        # Accumulate batch loss
        running_loss += loss.item() * x_batch.size(0)

    # Average loss for the epoch

```

```

epoch_loss = running_loss / len(train_loader.dataset)

# Print every 10 epochs
if (epoch+1) % 10 == 0:
    print(f"Epoch {epoch+1}/{epochs}, Loss: {epoch_loss:.4f}")

```

8.6 Monte Carlo Sampling

```

mc_samples = 2000 # Number of stochastic forward passes
model.eval()
predictions = []

with torch.no_grad():
    for _ in range(mc_samples):
        # Each pass samples new weights
        pred, _ = model(x_test_tensor.to(device))
        predictions.append(pred.cpu().numpy())

# Stack predictions into array
predictions = np.stack(predictions, axis=0)

# Compute mean and std across samples
y_pred_mean = predictions.mean(axis=0).squeeze()
y_pred_std = predictions.std(axis=0).squeeze()

# Unstandardize predictions to original scale
y_pred_original = y_pred_mean * y_std + y_mean

```

8.7 Gaussian Process Model

```

library(tidyverse)
library(caret)
library(ggplot2)
library(kernlab)

df <- read.csv("exam1.csv")

# Rename for convenience (optional)
colnames(df) <- c("distance", "error")

set.seed(538)

train_index <- createDataPartition(df$error, p = 0.7, list = FALSE)
train <- df[train_index, ]
test <- df[-train_index, ]

```

```

rmse <- function(y, yhat) sqrt(mean((y - yhat)^2))
mse  <- function(y, yhat) mean((y - yhat)^2)
r2   <- function(y, yhat) 1 - sum((y - yhat)^2) / sum((y - mean(y))^2)

gp_model <- kernlab::gausspr(
  error ~ distance,
  data = train,
  kernel = "rbfdot",
  var = 0.01
)

gp_pred <- predict(gp_model, test)
gp_res  <- test$error - gp_pred

gp_r2   <- r2(test$error, gp_pred)
gp_mse  <- mse(test$error, gp_pred)
gp_rmse <- rmse(test$error, gp_pred)

plot(
  gp_pred, gp_res,
  xlab = "Predicted values (GP)",
  ylab = "Residuals",
  main = "Gaussian Process Residuals"
)
abline(h = 0, lwd = 2)

plot(df$distance, df$error, main = "Gaussian Process Prediction",
  xlab = "Distance between missile and drone (km)",
  ylab = "Error in estimated distance (m)")
points(test$distance, gp_pred, col = "blue")
lines(test$distance, gp_pred, col = "red", lwd = 2)

ggplot(plot_df, aes(x = x)) +
  geom_point(aes(y = y), alpha = 0.5) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.2) +
  geom_line(aes(y = mean), linewidth = 1) +
  labs(
    x = "x",
    y = "y",
    title = "Gaussian Process with 95% Uncertainty Band"
  ) +
  theme_minimal()

results <- data.frame(
  Model = c("Gaussian Process"),

```

```

R2      = c(gp_r2),
MSE     = c(gp_mse),
RMSE    = c(gp_rmse)
)

print(results)

set.seed(538)

# Check the data

cat("Number of observations:", nrow(data), "\n")
cat("Distance range:", format(range(data$Distance), scientific = FALSE), "\n")
cat("Error range:", round(range(data$Error), 4), "\n\n")

# Visualization

p1 <- ggplot(data, aes(x = Distance, y = Error)) +
  geom_point(alpha = 0.6, size = 1.5, color = "steelblue") +
  labs(title = "Raw Data: Distance vs Error",
       subtitle = paste(nrow(data), "observations"),
       x = "Distance (km)",
       y = "Error (m)") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"),
        plot.subtitle = element_text(hjust = 0.5, color = "gray40"))
print(p1)

# Bootstrapping

n_bootstrap <- 1000
bootstrap_predictions <- matrix(NA, nrow = n_bootstrap, ncol = nrow(pred_grid))

for (i in 1:n_bootstrap) {
  # Bootstrap sample
  bootstrap_indices <- sample(1:nrow(data), nrow(data), replace = TRUE)
  bootstrap_data <- data[bootstrap_indices, ]

  # Fit GP on bootstrap sample
  gp_bootstrap <- gausspr(Error ~ Distance,
                          data = bootstrap_data,
                          kernel = "rbfdot",
                          variance.model = TRUE)

  # Make predictions

```

```

  bootstrap_predictions[i, ] <- predict(gp_bootstrap, pred_grid)
}

# Calculate CI from bootstrap
pred_ci_lower <- apply(bootstrap_predictions, 2, quantile, probs = 0.025)
pred_ci_upper <- apply(bootstrap_predictions, 2, quantile, probs = 0.975)

# Create prediction data frame
pred_df <- data.frame(
  Distance = pred_grid$Distance,
  Pred_mean = pred_mean,
  Pred_lower_95 = pred_ci_lower,
  Pred_upper_95 = pred_ci_upper,
  Pred_sd = apply(bootstrap_predictions, 2, sd)
)

# Plot with intervals

p1 <- ggplot() +
  # 95% CI (from bootstrap)
  geom_ribbon(data = pred_df,
    aes(x = Distance,
        ymin = Pred_lower_95,
        ymax = Pred_upper_95),
    fill = "blue", alpha = 0.55) +

  # Original data points
  geom_point(data = data,
    aes(x = Distance, y = Error),
    color = "black", alpha = 0.4, size = 1.2) +

  # Labels and theme
  labs(title = "Gaussian Process Regression (gausspr)",
    subtitle = "With 95% bootstrap CI",
    x = "Distance between missile and drone (km)",
    y = "Error (m)",
    caption = paste("Model: RBF kernel | Bootstrap samples:", 1000)) +

  theme_minimal() +
  theme(
    plot.title = element_text(hjust = 0.5, size = 20, face = "bold"),
    plot.subtitle = element_text(hjust = 0.5, size = 15, color = "gray40"),
    plot.caption = element_text(hjust = 1, size = 15, color = "gray50"),
    axis.title = element_text(size = 18),
    axis.text = element_text(size = 15),

```

```

    panel.grid.minor = element_blank()
  )

print(p1)

```

8.8 BNN Rstan

```

data {
  // Number of observations
  int<lower=1> N;

  // Number of input features (Distance)
  int<lower=1> K;

  // Number of hidden units
  int<lower=1> H;

  // Standardized input matrix
  matrix[N, K] X;

  // Standardized response vector
  vector[N] y;
}

parameters {
  // Weights and biases for hidden layer
  matrix[H, K] W1;
  vector[H] b1;

  // Weights and bias for output layer
  vector[H] W2;
  real b2;

  // Observation noise
  real<lower=0> sigma;

  // Hierarchical scale parameters for weight regularization
  real<lower=0> sW1;
  real<lower=0> sW2;
}

model {
  vector[N] mu;

  // Hyperpriors to control weight magnitude

```

```

sW1 ~ normal(0, 1);
sW2 ~ normal(0, 1);

// Priors on network parameters
to_vector(W1) ~ normal(0, sW1);
b1 ~ normal(0, 1);
W2 ~ normal(0, sW2);
b2 ~ normal(0, 1);

// Prior on noise variance
sigma ~ exponential(1);

// Forward pass through neural network
for (n in 1:N) {
  vector[H] h = tanh(W1 * (X[n]') + b1);
  mu[n] = dot_product(W2, h) + b2;
}

// Likelihood
y ~ normal(mu, sigma);
}

generated quantities {
  // Posterior predictive samples
  vector[N] y_rep;

  for (n in 1:N) {
    vector[H] h = tanh(W1 * (X[n]') + b1);
    y_rep[n] = normal_rng(dot_product(W2, h) + b2, sigma);
  }
}

```

8.9 BNN R Code

```

# =====
# Load required packages
# =====
# rstan is used to perform full Bayesian inference using HMC (NUTS)
# caret is used only for train/test splitting
library(rstan)
library(caret)

# Enable faster compilation and parallel chains
rstan_options(auto_write = TRUE)
options(mc.cores = parallel::detectCores())

```



```

# =====
# Load nonlinear dataset
# =====
# Distance: distance between missile and drone (km)
# Error: tracking error (meters)
data_raw = read.csv("exam1.csv")
colnames(data_raw) = c("Distance", "Error")

# =====
# Train/Test split
# =====
# 80% training, 20% testing to evaluate out-of-sample performance
set.seed(538)
idx = createDataPartition(data_raw$Error, p = 0.8, list = FALSE)

train_raw = data_raw[idx, ]
test_raw = data_raw[-idx, ]

X_train = as.matrix(train_raw$Distance)
y_train = train_raw$Error

X_test = as.matrix(test_raw$Distance)
y_test = test_raw$Error

# =====
# Standardization (training data only)
# =====
# Prevents data leakage and improves sampling efficiency
x_mu = colMeans(X_train)
x_sd = apply(X_train, 2, sd)

y_mu = mean(y_train)
y_sd = sd(y_train)

X_train_s = scale(X_train, x_mu, x_sd)
X_test_s = scale(X_test, x_mu, x_sd)

y_train_s = (y_train - y_mu) / y_sd

# =====
# Data passed to Stan
# =====
# H = number of hidden units (chosen as a small value to avoid overfitting)
stan_data = list(

```

```

N = nrow(X_train_s),
K = ncol(X_train_s),
H = 3,
X = X_train_s,
y = as.numeric(y_train_s)
)

# =====
# Run Stan (HMC via NUTS)
# =====
# This step samples from the full posterior distribution
# using Hamiltonian Monte Carlo (No-U-Turn Sampler)
fit = stan(
  file = "bnn_exam1.stan",
  data = stan_data,
  chains = 4,
  iter = 4000,
  warmup = 2000,
  seed = 538,
  control = list(adapt_delta = 0.95, max_treedepth = 12)
)

# =====
# Posterior predictive inference
# =====
# Posterior samples of the network parameters are propagated
# through the neural network to obtain predictive distributions
post = rstan::extract(fit)

S = length(post$sigma)
Ntest = nrow(X_test_s)
H = stan_data$H

mu_test = matrix(NA, S, Ntest)

for (s in 1:S) {
  Hmat = tanh(
    X_test_s %*% t(post$W1[s,]) +
    matrix(post$b1[s,], Ntest, H, byrow = TRUE)
  )
  mu_test[s,] = Hmat %*% post$W2[s, ] + post$b2[s]
}

# =====
# Unstandardize predictions

```

```

# =====
# Convert predictions back to the original scale
mu_mean = apply(mu_test, 2, mean) * y_sd + y_mu
mu_lo    = apply(mu_test, 2, quantile, 0.025) * y_sd + y_mu
mu_hi    = apply(mu_test, 2, quantile, 0.975) * y_sd + y_mu

# =====
# Model evaluation
# =====
# RMSE is computed on the test set
rmse = sqrt(mean((y_test - mu_mean)^2))
cat("BNN RMSE:", round(rmse, 3), "\n")

# =====
# Linear regression baseline
# =====
# Used to highlight the advantage of nonlinear modeling
lm_fit = lm(y_train ~ as.numeric(X_train))

lm_pred = predict(
  lm_fit,
  newdata = data.frame(x_train_num = as.numeric(X_test))
)

lm_rmse = sqrt(mean((y_test - lm_pred)^2))

```

References

- Beck, C. and Eldan, R. (2020). The geometry of neural network loss surfaces via random matrix theory. *Journal of Machine Learning Research*.
- Blundell, C., Cornebise, J., Kavukcuoglu, K., and Wierstra, D. (2015). Weight uncertainty in neural network. In *International conference on machine learning*, pages 1613–1622. PMLR.
- de Silva, T. Q. H. (2021). Variational autoencoders explained.
- Kumar, S. (n.d.). Pytorch bayesian convolutional neural networks. GitHub repository.
- Kunc, V. and Kléma, J. (2024). Three decades of activations: A comprehensive survey of 400 activation functions for neural networks. *arXiv preprint arXiv:2402.09092*.
- PyTorch Contributors (n.d.). Pytorch optimization algorithms (adam optimizer). Official documentation.